Upkeep Project Plan

Hey team! This is our plan for Upkeep from now until the tech summit in January: what we're building, how the pieces fit together, who owns what, and what's due each week. Please give it a read before Monday. If anything looks off or you'd do it differently, tell me. I'd much rather change it now than in week 5.

Team: Nmesoma (lead dev), Satujan (data, gust features & evaluation), Varun (health models), and Karim (alerts & dashboard).

What we're building

Upkeep is an early-warning system for wind turbines. It replays real turbine sensor data as if it were live, learns how each part of a healthy turbine should behave in any wind, and warns when a part starts running hotter than it should, days or weeks before it breaks. Every turbine shows as green, yellow or red, and every warning comes with its reasons ("gearbox bearing running 6°C hotter than expected for 3 days, after 14 high-gust periods this week"), so a maintenance crew knows what to check and why.

The gust angle is what makes Upkeep ours. Gusts are sudden bursts of wind that put sharp, uneven loads on the blades, gearbox and bearings. We measure gustiness in every 10-minute reading and ask one question all semester: **are periods of heavy gusts followed by parts wearing out faster?**

We use two kinds of models:

- **Normal-behaviour models (LightGBM)** that predict what a part's temperature should be from the wind, power output and outside temperature. The gap between real and expected (the **residual**) is our warning signal
- **An anomaly detector (Isolation Forest)** that looks at all the residuals together and flags combinations that don't look like a healthy turbine

The other half of the project is testing it properly. Our dataset includes real turbine faults with known dates, so every week we replay the months before each fault and check: did Upkeep warn us in time, and how many days ahead? We also replay healthy periods to count false alarms. At the summit, the goal is a live dashboard of a wind farm with turbines changing colour as the data plays, a turbine going yellow then red days before its real recorded fault, and a chart showing our scores going up over the semester.

What we're aiming for by the summit:

- Predict healthy part temperatures within 2°C on average
- Warn (yellow or red) before 70%+ of the recorded faults, with a median warning of at least 3 days ahead
- No more than 1 false red alert per turbine per month on healthy periods
- Every yellow or red alert says which part drifted, by how much, for how long, and what the recent gust exposure was, and reviewers agree the reasons make sense on 9 of 10 spot-checked alerts
- A clear, honest chart answering the gust question

- Each reading scored in under 1 second, so we can replay a month of a whole wind farm in under 5 minutes on stage

Scope

What we're doing:

- The CARE to Compare dataset, starting with Wind Farm A (5 turbines, 22 datasets, each a year of healthy data plus a test period, some ending in a real fault). It's public, anonymised operator data, free to download
- A readable sensor map, cleaning, a "normal operation only" filter, and a shared feature pipeline including gust features
- Normal-behaviour models for 2 to 3 key parts (gearbox bearing and generator bearing first), plus a fixed-temperature-limit baseline to beat
- An Isolation Forest across all residuals, and a health score per turbine with tuned green/yellow/red levels
- Plain-English reasons for every yellow or red alert
- A replay simulator that plays the data back as if live
- An eval script that scores every model on the recorded faults and healthy periods weekly, and a gust study
- A Streamlit dashboard with a wind farm overview, a turbine detail page and a scorecard page

Nice to have if we get time: adding Wind Farm B, the dataset's official CARE score, estimating days until failure, an LSTM model, a "what if" panel where visitors drag wind and gust sliders, running the eval on every PR with GitHub Actions, a hosted demo.

Not doing: real turbine hardware or physical sensors, physics simulations of turbines, connecting to any real wind farm, a mobile app, or controlling turbines (Upkeep warns, a human decides).

Tools

Here's the default stack. Nmesoma can swap anything if there's a good reason, just add a line to DECISIONS.md so we all know why.

Part	What we're using	Notes
Language	Python 3.11	Same version for everyone
Code + tasks	GitHub repo `upkeep`, Jira	One branch per ticket, PRs reviewed by Nmesoma
Setup	`venv` + `requirements.txt`	Everything runs locally
Data	`pandas`, `pyarrow` (Parquet files)	Time-indexed per turbine
Normal-behaviour models	`lightgbm`	One model per part; linear regression version as a sanity check

Part	What we're using	Notes
Classic ML	`scikit-learn`	Isolation Forest, linear regression, metrics
Saving models	`joblib`	Saved models shared through the Drive folder, not GitHub
Eval	Our own `run_eval.py`	Results saved per run as CSV/JSON
App	Streamlit + Plotly	Farm overview, turbine detail, scorecard
Alert sentences	Templates	Fill-in-the-blank from the reasons
Secrets	`.env` (never committed) + `.env.example`	`python-dotenv`, probably unused
Tests	`pytest`	

Everything here is free and runs on a normal laptop, so we don't need a budget.

Data

We're using **CARE to Compare**, a public wind turbine fault dataset on Zenodo (CC BY-SA 4.0 licence). It's one 5.5 GB download, no account needed, and the turbines are anonymised.

- Readings every 10 minutes from each turbine: wind speed, power, rotor speed, outside temperature, and temperatures of parts like the gearbox and generator bearings. Many sensors include the average, minimum, maximum and spread for each 10 minutes
- Wind Farm A has 5 turbines and 22 datasets. Each dataset is about a year of training data plus a test period. Some test periods lead up to a real recorded fault (the event table gives its dates), others are healthy for comparison
- A status column says whether the turbine was running normally, stopped or in service. We train only on normal operation

A few rules:

- The download never goes in GitHub. Everyone downloads it from Zenodo into `data/raw/`, and the README says exactly how
- **Keep the dataset's own split.** Train on each dataset's training year, validate on the last 2 months of it, test on its test period. Never shuffle, so we're always predicting the future like a real system would
- The test periods are locked. Nobody tunes anything on them; tuning happens on validation
- Fault dates and status codes are never model inputs. If a model ever looks too good, we check for leakage before we celebrate
- **Sensor map:** the columns have anonymous names, so in week 1 Satujan writes `SENSORS.md`, mapping each column we use to a readable name using the dataset's feature description file. Everyone uses the readable names

- **Gust columns check:** in week 1, Satujan confirms which wind speed max and spread columns Wind Farm A has. If they're missing or patchy, we run the gust study on the public Kelmarsh wind farm dataset instead, which records 10-minute wind speed min, max and spread

Seed sample: in week 1, Satujan commits 3 Wind Farm A datasets (2 that end in a fault, 1 healthy), already renamed to readable columns, to `data/seed/` as Parquet, plus a first version of the features for them. That way everyone can start building right away instead of waiting on the full pipeline. If it's not in by Thursday, Nmesoma will add it.

Gust study: Satujan also owns the analysis that answers our big question. For each turbine, compare gust exposure in the days before residuals start rising against normal stretches, and chart it. It also feeds the gust context in every alert.

Who's doing what

Everyone owns their own folder, one of the handoff formats, and one score, so it's always clear whose part something is. Expect around 4 to 6 hours a week including the Monday meeting, and less during exams.

	Role	Owns	Your score	Backup
Nmesoma	Lead dev	`stream/`, `health/`, `stubs.py`, `CONTRACTS.md`, code review, connecting everything	Faults warned and how early, false alarms, live speed	Karim
Satujan	Data, gust features & evaluation	`data/`, `features/`, `SENSORS.md`, `FEATURES.md`, `eval/`, the gust study	Eval posted every Friday, no leakage found, gust question answered	Varun
Varun	Health models	`models/`, `predictions/`	Temperature error on healthy periods (target under 2°C)	Satujan
Karim	Alerts & dashboard	`explain/`, `app/`	Every alert has reasons; reasons make sense on 9/10 spot checks	Nmesoma

Nmesoma: sets up the repo rules, `CONTRACTS.md`, the stubs and the replay simulator in week 1. Then the live pipeline (`score_reading()`), the health score and the green/yellow/red rules, and keeping the whole thing running end to end. You'll swap in the real pieces as they land, keep `main` working, and review PRs. If someone gets stuck on something critical you might help out, but cap that at about 2 hours a week. Past that, it's on me to cut scope.

Satujan: download CARE, the sensor map, cleaning and the normal-operation filter, the seed sample, `make_features()` with the gust features, and `FEATURES.md`. You also own `run_eval.py` and the fault-event checks, so the people building models aren't grading their own homework, and the gust study.

You'll run the eval every Friday and post the numbers in the chat. Once features are stable around week 5, your build load drops, so you'll help Karim with the scorecard and the gust chart.

Varun: the fixed-limit baseline and a linear regression version first (so we know what "good" means), then LightGBM normal-behaviour models for the gearbox bearing and generator bearing, and a third part if the data supports it. Then the Isolation Forest across all the residuals. Report which inputs matter most to each model. Trying an LSTM is yours if we have time.

Karim: the reasons for each alert (which part, how far off, how long), the gust context line, the templates that turn reasons into one plain sentence, and all three dashboard pages. The "what if" sliders panel is yours if we have time.

How it works

There are three parts, and they only connect in two places: the feature file and the `score_reading()` function. Each person mostly works inside their own boxes.

1. **Train the models (weekly):** load the data, keep normal operation only, build features with the shared `make_features()`, then each model trains on the training year, gets tuned on validation, and saves itself plus its predictions file.
2. **Score live (the demo):** the replay simulator plays test readings in order, fast-forwarded. For each one: build its features, get the expected temperature of each part, work out the residuals, get the anomaly score, combine them into a health score and colour, and if it's yellow or red, attach the reasons and a one-sentence explanation. Every result is written to `health.jsonl`, and the dashboard reads that file and refreshes every second.
3. **Test it (weekly):** run every model on the locked test periods, check which faults got a warning and how early, count false alarms on healthy periods, save the results with the date, and chart them on the scorecard page.

Colours only change when a problem lasts. A part has to stay well above its expected temperature for a few hours before a turbine goes yellow, so one odd reading never triggers an alert. Nmesoma tunes those rules on validation.

Handoff formats

This is the part that keeps us from waiting on each other. We lock these five formats in week 1 and Nmesoma writes them up in `CONTRACTS.md`. If you need to change one later, open a PR so everyone sees it before it lands.

1. Reading (one 10-minute row, everyone reads it): the columns we use, with readable names from `SENSORS.md`


```
{"reading_id": "A-T3-2022-03-14T06:20", "turbine_id": "A-T3", "dataset_id": "A-68",
"timestamp": "2022-03-14T06:20:00", "wind_speed_avg": 11.2, "wind_speed_max": 17.9,
"wind_speed_std": 2.4, "power_kw": 1830.0, "rotor_rpm": 15.1, "ambient_temp": 9.5,
"gearbox_bearing_temp": 64.2, "generator_bearing_temp": 58.7, "status": "normal"}
```

2. Features (Satujan owns, everyone reads): `make_features(df) -> df` takes readings for one turbine sorted by time and returns one row per `reading_id` with the columns listed in `FEATURES.md`. Saved as `data/processed/features_<dataset>.parquet`.

```
def make_features(readings: pd.DataFrame) -> pd.DataFrame:
    # one row per reading_id, e.g. wind_speed_avg, power_kw, ambient_temp,
    # gust_factor (max / avg), turbulence (std / avg), wind_jump (change from last
reading),
    # gusts_last_24h, gusts_last_7d, temp_1h_ago, hour_of_day, ...
```

3. Model (Varun owns, Nmesoma and Satujan use them): every normal-behaviour model is a class with the same four methods, and writes a predictions file after each run. The Isolation Forest follows the same shape with `predict_score()` instead.

```
class NormalModel:
    def fit(self, X, y): ...                  # y = one part's temperature
    def predict(self, X) -> np.ndarray: ...  # expected temperature in °C
    def save(self, path): ...
    @classmethod
    def load(cls, path): ...
# predictions/<model>_<part>_<dataset>.csv  ->  reading_id, actual, expected,
residual
```

4. `explain()` (Karim owns, Nmesoma calls it)

```
def explain(recent: pd.DataFrame, residuals: dict, gusts: dict) -> dict:
    # {"reasons": [{"part": str, "residual_c": float, "hours": float, "text": str}],
# top 3
    #  "gust_context": str, "sentence": str}
```

5. Health status (Nmesoma writes, the dashboard and eval read): `score_reading(reading) -> dict`, one per line in `data/live/health.jsonl`

```
{"reading_id": "A-T3-2022-03-14T06:20", "turbine_id": "A-T3", "timestamp": "2022-03-
14T06:20:00", "health": "yellow", "health_score": 0.64, "residuals":
{"gearbox_bearing": 6.1, "generator_bearing": 1.2}, "anomaly_score": 0.58,
"gusts_last_7d": 14, "reasons": [{"part": "gearbox_bearing", "residual_c": 6.1,
"hours": 72, "text": "Gearbox bearing 6°C hotter than expected for 3 days"}],
"sentence": "Gearbox bearing has run 6°C hotter than expected for 3 days, after a
gusty week.", "fault_event": "A-68", "days_to_fault": 5.3, "latency_ms": 120}
```

`fault_event` and `days_to_fault` are only there so the dashboard and eval can show how early a warning came. The models never see them.

Stubs: Nmesoma also commits `src/upkeep/stubs.py` in week 1 with a fake `make_features()` (a few simple columns), fake models (return fixed temperatures), a fake `explain()` (canned reasons) and a fake `score_reading()`. Build against those until the real ones are ready, then it's a one-line swap.

GitHub

One repo, `upkeep`, in the WCS AI org. I'll send invites, and Nmesoma has Maintain access to set things up.

```
upkeep/
├── README.md                how to install, download CARE and run (Nmesoma)
├── CONTRACTS.md             the 5 formats above (Nmesoma)
├── DECISIONS.md             one line per decision + why (Nmesoma)
```

```
├── SENSORS.md                column code names -> readable names (Satujan)
├── FEATURES.md               every feature and what it means (Satujan)
├── requirements.txt
├── .env.example              template; real .env never gets committed
├── data/
│   ├── raw/                  CARE download, never committed (Satujan)
│   ├── seed/                 3 datasets + their features (Satujan)
│   ├── processed/            cleaned data + feature files, not committed (Satujan)
│   └── live/health.jsonl     written by the live pipeline (Nmesoma)
├── src/upkeep/
│   ├── stubs.py              fake versions of everything (Nmesoma)
│   ├── data/                 load.py, sensors.py, clean.py (Satujan)
│   ├── features/             features.py, gusts.py (Satujan)
│   ├── models/
│   │   ├── baseline_rule.py  fixed temperature limit (Varun)
│   │   ├── linear_nbm.py     linear regression check (Varun)
│   │   ├── lgbm_nbm.py       LightGBM normal-behaviour models (Varun)
│   │   └── iforest.py        Isolation Forest on residuals (Varun)
│   ├── health/               score.py, levels.py (Nmesoma)
│   ├── explain/              reasons.py, sentence.py (Karim)
│   └── stream/               replay.py, live.py, score_reading() (Nmesoma)
├── predictions/              one CSV per model per part per dataset (Varun)
├── eval/
│   ├── run_eval.py           scores every model + the health levels (Satujan)
│   ├── events.py             fault dates, warning lead times (Satujan)
│   ├── gust_study.py         the gust question (Satujan)
│   └── results/              one dated file per run (Satujan)
├── app/
│   ├── farm.py               wind farm overview (Karim)
│   ├── turbine.py            one turbine: real vs expected + reasons (Karim)
│   └── scorecard.py          weekly scores + gust chart (Karim)
└── tests/                    pytest tests (everyone, for their own folder)
```

Saved model files go in `models_saved/` (ignored by git) and get shared through the Drive folder.

A few rules:

- `main` is protected, so no pushing to it directly
- Branch per Jira ticket, named with the ticket number (e.g. `UPK-12-seed-sample`)
- Every PR needs 1 approval. Nmesoma reviews everyone, and Varun reviews Nmesoma's
- In your PR, link the ticket and say how to test it
- `main` should always run. If your merge breaks it, please fix it that day
- Never commit the raw data, saved models or a `.env`. If one gets committed by accident, tell Nmesoma right away
- PRs get reviewed within 48 hours. If Nmesoma's busy, anyone can approve

If you're blocked

Nobody should ever be sitting around waiting on someone else's part. Here's how we avoid that:

- **Build against the stubs and the seed sample.** You don't need the real pieces to start.
- **We'll have a rough version running end to end by the end of week 2.** After that, everyone's just improving their own part.

- **Grab a backup-lane ticket.** Everyone has a couple in Jira (label `backup-lane`) that don't depend on anyone. If you're stuck waiting, pick one up and let me know in the chat.

	Your backup-lane tasks
Nmesoma	Speed up the replay on the seed sample, README and setup docs, catch up on reviews
Satujan	Sketch new candidate gust features, write tests for `make_features()`, check more sensors in `SENSORS.md`
Varun	Try model settings on the seed sample, plot which inputs matter, tests for your models
Karim	Build dashboard pages against the stub health file, write more sentence templates

If something stalls: if a ticket goes 4 days with no activity, I'll check in with you, no stress. If I don't hear back within a day, I'll move it to your backup (or Nmesoma if it's blocking the week's goal). If it's still stuck after a week, Nmesoma and I will shrink it, push it to later, or cut it. This isn't about getting anyone in trouble. I just don't want one thing holding the whole project up.

The six things the summit can't happen without are the seed sample, `make_features()`, the LightGBM normal-behaviour model, `score_reading()`, the wind farm page and `run_eval.py`. Everything else, including the Isolation Forest and the gust study, can be trimmed if we need to. (The gust study is what makes Upkeep ours, so I really hope we don't have to.)

Timeline

The plan is to have the core built and frozen by **Dec 6**, so exams don't wreck us. December is just small fixes, and January is polish and rehearsal. The summit date isn't announced yet, so this assumes it's after Jan 17. I'll adjust once we know.

Work is due Friday each week (PR merged, Jira ticket in Done), and we look at it together at the Monday meeting. The weeks marked **checkpoint** are when we stop and make sure we're on track.

Week	Nmesoma	Satujan	Varun	Karim	We check
1 Oct 26-Nov 1	`CONTRACTS.md`, stubs, repo rules, replay v0	CARE downloaded, `SENSORS.md`, gust columns check, seed sample + seed features, `FEATURES.md`	Fixed-limit baseline + linear model on the seed sample	Wind farm page v0 on stub health data	Files are in the repo
2 Nov 2-8	`score_reading()` v0 writing `health.jsonl`	All Wind Farm A datasets cleaned, normal-only filter, `make_features()` v1 with gust	LightGBM model for the gearbox bearing +	Turbine page: real vs expected temperature	**Checkpoint:** replay runs, turbines change

Week	Nmesoma	Satujan	Varun	Karim	We check
		features, `run_eval.py` v0	predictions file	chart	colour, first scores
3 Nov 9-15	Real models plugged in, health colours v1 (persistence rule)	Gust exposure features (last 24h / 7 days), fault event table in the eval	Generator bearing model (+ a third part if possible), tuning on validation	Reasons v1 and sentence templates	Week 3 scores
4 Nov 16-22	Health score combining parts + anomaly score, colour levels tuned on validation, latency logging	Leakage review, warning lead-time metric, gust study v1	Isolation Forest on residuals, feature importance report	Gust context in alerts, scorecard v1	Which faults we caught and how many days early
5 Nov 23-29	Connect everything, fix top misses and false alarms	Features frozen, full eval	Lock final models	Spot-check 20 alerts' reasons	**Checkpoint:** full scorecard
6 Nov 30-Dec 6	Under 1 second per reading, a month in under 5 minutes, handles missing sensor values	One-command retrain + eval, gust study final chart, help with scorecard	Document final setup	Dashboard polish	**Checkpoint:** 10-min demo to WCS execs, then freeze
7-8 Dec 7-20	Fix top 2 issues	Weekly eval only	Fix top 2 issues	Fix top 2 issues	Scores aren't dropping
Break	Optional	Optional	Optional	Optional	
9 Jan 4-10	Demo mode: curated replay of our best early warning	Final eval + summit charts	Slide on how normal-behaviour models work	Final dashboard polish	Full run-through as a visitor
10 Jan 11-17	Demo script, laptop setup	Backup demo video	Poster on how Upkeep works	Dashboard on the big screen	**Checkpoint:** 2 rehearsals done

Our week

- **Monday 6:30-8:30 pm, club meeting:** the first 30 minutes we go through what everyone built, look at the scores and anything that's stuck, and start the new sprint. The rest is work time together.
- **Thursday:** a quick 3-line update in the chat (what you finished, what's next, anything blocking you).
- **Friday:** your week's work is merged and Satujan posts the scores.
- **Anytime:** PRs reviewed within 48 hours, and hop on a call with whoever you need.

Jira: everything lives in the Upkeep space. Each ticket has a "Done when" line so it's clear when it's finished, and the labels let you filter by role (`lead`, `data`, `models`, `dashboard`), week (`week-03`) or `backup-lane`.

What could go wrong

If this happens	We do this
The 5.5 GB download is too slow or too big for someone's laptop	Build on the seed sample; only Satujan and whoever runs final training need the full download
The anonymous sensor names are confusing	`SENSORS.md` in week 1, and only the columns we actually need
Wind Farm A doesn't have good gust columns	Run the gust study on the Kelmarsh dataset instead
Too many false alarms	Persistence rule, colour levels tuned on validation, smoother residuals
Scores look too good to be true	Leakage review in week 4, drop any input that gives the answer away
Gusts show no link to failures	That's still a real finding; we show it honestly and lean on the early warnings for the demo
Someone gets swamped	Backup lanes, buddies, and cutting non-essential stuff
The demo laptop chokes live	Pre-computed demo replay + a backup video
Exams	Everything major is done by Dec 6

Still figuring out

- ☐ Summit date (waiting on WCS)
- ☐ Which 2 to 3 parts we model, based on which sensors are reliable (Satujan and Varun, end of week 1)
- ☐ Whether the gust study uses Wind Farm A or Kelmarsh (Satujan, end of week 1)

- ☐ How the health score combines parts: worst part wins, or a weighted mix with the anomaly score (Nmesoma, week 4)
- ☐ Whether we add Wind Farm B (Nmesoma, week 5)

Really excited to build this with you all. See you Monday!

Lianna